Unit 2 — Self Study Topics

Unit: 2 | **Folder:** self-study | **CO:** CO2, CO3, CO4

This file covers the self-study topics designated for Unit 2 of the ADSA course. Each section is independent and can be studied after completing the corresponding unit lectures.

Topic 1 — Topological Sorting (In-Depth)

What is Topological Sort?

A **topological ordering** of a DAG (Directed Acyclic Graph) is a linear ordering of vertices such that for every directed edge u → v, vertex u appears before v.

Only valid for **DAGs** — cyclic graphs have no topological ordering.

Algorithm 1: DFS-Based (Reverse Post-Order)

```
from collections import defaultdict

def topological_sort_dfs(V, edges):
    """DFS topological sort. O(V + E)"""
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)

    visited = set()
    stack = []

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)
        stack.append(node)   # Add to stack AFTER visiting all descenda

    for v in range(V):
        if v not in visited:
            dfs(v)

    return stack[::-1]   # Reverse of post-order = topological order
```

Algorithm 2: Kahn's BFS

(Covered in E10 — Course Schedule)

Multi-Source Topological Sort

```
def all_topological_sorts(graph, in_degree, V):
    """Generate ALL valid topological orderings (backtracking). O(V! ×
    result, path = [], []
    available = [i for i in range(V) if in_degree[i] == 0]

    def backtrack():
        if len(path) == V:
            result.append(path[:])
            return
        for node in available[:]:
            path.append(node)
            available.remove(node)
            for neighbor in graph[node]:
                in_degree[neighbor] -= 1
                if in_degree[neighbor] == 0:
                    available.append(neighbor)
            backtrack()
            # Undo
            path.pop()
            for neighbor in graph[node]:
                if in_degree[neighbor] == 0:
                    available.remove(neighbor)
                in_degree[neighbor] += 1
            available.append(node)

    backtrack()
    return result
```

Applications

- Build systems (Makefile, Gradle)
- Package dependency resolution (pip, npm)
- Course prerequisite scheduling
- Compiler symbol resolution

Topic 2 — Strongly Connected Components (SCC)

Definition

A **Strongly Connected Component** of a directed graph is a maximal set of vertices such that there is a path from each vertex to every other vertex in the set.

Kosaraju's Algorithm

```
1. Perform DFS on original graph; push vertices to stack in finish orde
2. Transpose the graph (reverse all edges)
3. Perform DFS on transposed graph in order of stack (largest finish ti
4. Each DFS tree in step 3 = one SCC
```

```
def kosaraju(V, edges):
    """Find SCCs using Kosaraju's algorithm. O(V + E)"""
```

```
    graph = defaultdict(list)
    transposed = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        transposed[v].append(u)

    # Step 1: DFS order
    visited = [False] * V
    finish_stack = []

    def dfs1(node):
        visited[node] = True
        for nb in graph[node]:
            if not visited[nb]:
                dfs1(nb)
        finish_stack.append(node)

    for v in range(V):
        if not visited[v]:
            dfs1(v)

    # Step 2: DFS on transposed graph
    visited = [False] * V
    sccs = []

    def dfs2(node, component):
        visited[node] = True
        component.append(node)
        for nb in transposed[node]:
            if not visited[nb]:
                dfs2(nb, component)

    while finish_stack:
        node = finish_stack.pop()
        if not visited[node]:
            component = []
            dfs2(node, component)
            sccs.append(component)

    return sccs
```

Tarjan's Algorithm (Single Pass)

Uses DFS with **discovery time** and **low-link value** to find SCCs in one pass.

- `disc[u]` = discovery time of u
- `low[u]` = lowest discovery time reachable from subtree rooted at u
- If `low[u] == disc[u]`, u is the root of an SCC → pop from stack

Time: O(V + E) — more efficient in practice than Kosaraju for a single pass.

Applications

- Web page ranking (SCC = equivalent pages)
- Compiler optimization (identifying mutually recursive functions)
- Social network analysis (tightly-knit communities)

Topic 3 — Floyd-Warshall Algorithm

Problem

Find shortest paths between **all pairs** of vertices in a weighted directed graph (negative edges allowed, but no negative cycles).

Algorithm

```
def floyd_warshall(V, edges):
    """
    All-pairs shortest paths.
    Time: O(V³)  |  Space: O(V²)
    """
    INF = float('inf')
    dist = [[INF] * V for _ in range(V)]
    for v in range(V):
        dist[v][v] = 0
    for u, v, w in edges:
        dist[u][v] = w

    # Try each vertex k as an intermediate node
    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    # Detect negative cycles
    for v in range(V):
        if dist[v][v] < 0:
            raise ValueError(f"Negative cycle detected at vertex {v}")

    return dist
```

Trace (4 nodes, k = 0..3)

```
Initial:          After k=0:        After k=1:
    0  1  2  3          0  1  2  3
0 [ 0  3  ∞  7 ]   0 [ 0  3  ∞  7 ]   0 [ 0  3  ∞  7 ]
1 [ 8  0  2  ∞ ]   1 [ 8  0  2 15]   1 [ 8  0  2 15]
2 [ 5  ∞  0  1 ]   2 [ 5  8  0  1 ]   2 [ 5  8  0  1 ]
3 [ 2  ∞  ∞  0 ]   3 [ 2  5  ∞  0 ]   3 [ 2  5  4  0 ]
```

Comparison with Dijkstra

Property	Floyd-Warshall	Dijkstra (all pairs)

Negative edges	Yes	No
Time	$O(V^3)$	$O(V(V+E) \log V)$
Space	$O(V^2)$	$O(V + E)$ per run
Best for	Dense graphs, negative edges	Sparse graphs

Topic 4 — Ford-Fulkerson Network Flow

Problem

Given a directed graph with **capacities** on edges, find the **maximum flow** from a source s to a sink t.

Key Concepts

- **Residual graph:** For each edge u→v with capacity c and flow f, add:
  - Forward edge u→v with residual capacity $c - f$
  - Backward edge v→u with capacity f (allows undoing flow)
- **Augmenting path:** Any path from s to t in the residual graph
- **Max-flow Min-cut Theorem:** max flow = min cut capacity

```
from collections import deque

def bfs_find_path(graph, s, t, parent):
    """BFS to find augmenting path in residual graph (Edmonds-Karp)."""
    visited = {s}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if v not in visited and graph[u][v] > 0:
                visited.add(v)
                parent[v] = u
                if v == t:
                    return True
                queue.append(v)
    return False

def edmonds_karp(graph, s, t):
    """
    Edmonds-Karp (BFS Ford-Fulkerson).
    Time: O(VE²)  |  Space: O(V + E)
    """
    max_flow = 0
    parent = {}

    while bfs_find_path(graph, s, t, parent):
        # Find minimum residual capacity along the path
        path_flow = float('inf')
        v = t
```

```
        while v != s:
            u = parent[v]
            path_flow = min(path_flow, graph[u][v])
            v = u

        # Update residual capacities
        v = t
        while v != s:
            u = parent[v]
            graph[u][v] -= path_flow
            graph[v][u] = graph.get((v, u), 0) + path_flow
            v = u

        max_flow += path_flow
        parent = {}

    return max_flow
```

Applications

- Network bandwidth allocation
- Bipartite matching (job assignment, marriage problem)
- Image segmentation
- Airline scheduling

Topic 5 — Advanced Graph Coloring

k-Coloring Problem

Assign colors to graph vertices such that no two adjacent vertices share a color, using at most k colors.

- **2-coloring** = bipartite check (BFS/DFS, O(V + E))
- **3-coloring** = NP-complete
- **Chromatic number χ(G)** = minimum k for valid coloring

Greedy Coloring

```
def greedy_coloring(V, adj):
    """Greedy graph coloring (not always optimal). O(V + E)"""
    color = [-1] * V

    for v in range(V):
        # Find colors used by neighbors
        neighbor_colors = {color[u] for u in adj[v] if color[u] != -1}
        # Assign smallest available color
        c = 0
        while c in neighbor_colors:
            c += 1
        color[v] = c
```

```
    return color, max(color) + 1   # assignments, num_colors used
```

Greedy gives at most $\Delta(G)+1$ colors (Δ = max degree). Not always optimal.

Applications

- Register allocation in compilers
- Timetable scheduling (courses with shared students)
- Frequency assignment in radio networks
- Map coloring (4-color theorem: every planar graph is 4-colorable)

Topic 6 — Perfect Hashing

Problem

Standard hash tables have O(1) **average** lookup. **Perfect hashing** achieves O(1) **worst-case** lookup for a static set of keys.

Two-Level Scheme (Fredman-Komlós-Szemerédi)

```
Level 1: Hash n keys into m = n buckets using h₁
Level 2: For each bucket i with bᵢ keys, use a second hash table of size
         (guaranteed no collisions with high probability)

Space: O(n) expected
Build time: O(n) expected
Lookup: O(1) worst case
```

```
class PerfectHashTable:
    def __init__(self, keys):
        self.n = len(keys)
        self.primary = self._build(keys)

    def _build(self, keys):
        """Build two-level perfect hash table."""
        import random
        p = 10**9 + 7   # Large prime

        # Level 1: hash into n buckets
        while True:
            a1 = random.randint(1, p-1)
            b1 = random.randint(0, p-1)
            buckets = [[] for _ in range(self.n)]
            for k in keys:
                buckets[((a1 * k + b1) % p) % self.n].append(k)

            # Level 2: for each bucket, build perfect table of size bᵢ²
            tables = []
            for bucket in buckets:
                bi = len(bucket)
                if bi == 0:
```

```
                tables.append(None)
                continue
            size = bi * bi
            # Find collision-free hash for this bucket
            while True:
                a2 = random.randint(1, p-1)
                b2 = random.randint(0, p-1)
                slots = [None] * size
                ok = True
                for k in bucket:
                    slot = ((a2 * k + b2) % p) % size
                    if slots[slot] is not None:
                        ok = False; break
                    slots[slot] = k
                if ok:
                    tables.append((slots, a2, b2, p, size))
                    break

        return (buckets, tables, a1, b1, p)
```

Topic 7 — Double Hashing

Concept

Double hashing is an open addressing collision resolution strategy where the probe sequence uses **two independent hash functions**:

$probe(i) = (h_1(k) + i \times h_2(k)) \bmod m$

Requirements:

- $h_2(k)$ must never be 0 (would cause infinite loop)
- $h_2(k)$ must be coprime to m (typical: m is prime)

```
class DoubleHashTable:
    def __init__(self, m=11):
        self.m = m          # Table size (prime)
        self.table = [None] * m
        self.DELETED = object()

    def h1(self, key):
        return key % self.m

    def h2(self, key):
        return 1 + (key % (self.m - 1))   # Never returns 0

    def insert(self, key):
        for i in range(self.m):
            slot = (self.h1(key) + i * self.h2(key)) % self.m
            if self.table[slot] is None or self.table[slot] is self.DEL
                self.table[slot] = key
                return
```

```
        raise Exception("Table is full")

    def search(self, key):
        for i in range(self.m):
            slot = (self.h1(key) + i * self.h2(key)) % self.m
            if self.table[slot] is None:
                return False
            if self.table[slot] == key:
                return True
        return False
```

Advantages over Linear/Quadratic Probing

- Eliminates **primary clustering** (linear probing) and **secondary clustering** (quadratic probing)
- Distributes keys more uniformly
- Expected probes: $1/(1-\alpha)$ for successful, $1/(1-\alpha)^2$ for unsuccessful (similar to random hashing)

Topic 8 — Extendible Hashing

Motivation

Standard hash tables require **full rehashing** when load factor exceeds threshold — O(n) time. Extendible hashing grows **incrementally**, rehashing only one bucket at a time.

Structure

```
Directory: array of 2^d pointers to buckets (d = global depth)
Buckets: physical pages of fixed size, each with local depth l_i

Global depth d = number of bits used to index the directory
Local depth l_i = number of bits that distinguish this bucket's keys
```

Split Operation

```
When a bucket B with local depth l overflows:
1. Increase local depth of B: l_i = l + 1
2. Create a new bucket B' with l_i = l + 1
3. If l < d: reroute half the directory entries from B to B', redistrib
4. If l == d: double the directory (d = d + 1), then split
```

Advantages

- No full rehashing — only one bucket splits at a time
- Directory doubles only when needed (amortized O(1) insert)
- Used in: PostgreSQL hash indexes, IBM DB2

Topic 9 — File Indexing

Dense vs Sparse Index

Type	Index entries	Space	Lookup
Dense	One per record	O(n)	Direct
Sparse	One per block	O(n/b)	Block scan
Multi-level	Hierarchical	O(log n) levels	Tree walk

Multi-Level Index

```
Primary index (sparse):  ←── on sorted data file
Secondary index (dense): ←── on non-ordering key fields
Clustering index:        ←── records physically sorted by field
Non-clustering index:    ←── records NOT sorted by field (like a book i
```

B+ Tree Index (Database)

```
class BPlusTreeIndex:
    """Conceptual B+ tree index used in RDBMS."""
    def range_query(self, low_key, high_key):
        """
        O(log n + k) range query:
        1. Tree traversal to leaf with low_key: O(log n)
        2. Leaf-level scan: O(k) records
        """
        leaf = self._find_leaf(low_key)
        results = []
        while leaf and leaf.keys[0] <= high_key:
            for key, ptr in zip(leaf.keys, leaf.ptrs):
                if low_key <= key <= high_key:
                    results.append(self._fetch_record(ptr))
            leaf = leaf.next   # Follow leaf linked list
        return results
```

Applications

- Database query optimization
- Library catalog systems
- File system directories (HFS+, NTFS use B-trees)

Topic 10 — B+ Tree Indexing in Databases

Database Index Anatomy

```
-- SQL creates a B+ tree index internally:
CREATE INDEX idx_student_grade ON students(grade);

-- Range query uses B+ tree:
SELECT * FROM students WHERE grade BETWEEN 80 AND 90;
-- → Tree walk to grade=80, then leaf scan to grade=90
```

Query Optimization

With index: O(log n + k) — tree walk + result fetch
Without index: O(n) — full table scan

Clustered vs Non-Clustered

```
Clustered index:
  Physical order of rows = index order
  → One clustered index per table
  → Range queries very fast (sequential disk reads)

Non-clustered index:
  Separate structure; leaf nodes hold row pointers
  → Multiple allowed per table
  → Random disk accesses for range queries
```

Index Selection Strategy

```
Create index when:
  - Column appears frequently in WHERE clauses
  - Column used in JOIN conditions
  - Selectivity is high (few duplicates)

Avoid index when:
  - Table is very small
  - Column has low selectivity (e.g., boolean flags)
  - Heavy INSERT/UPDATE workload (index maintenance overhead)
```

References

- Cormen et al., *Introduction to Algorithms*, 4th Ed. (MIT Press, 2022) — Ch. 22–25 (Graph Algorithms)
- Skiena, *The Algorithm Design Manual*, 3rd Ed. (Springer, 2020) — Ch. 7 (Graph Traversal)
- Ramakrishnan & Gehrke, *Database Management Systems*, 3rd Ed. — Ch. 8–12 (Indexing)
- Laaksonen, *Guide to Competitive Programming*, 3rd Ed. (Springer, 2024)